@poly-unified/sdk

A small TypeScript SDK prototype that unifies Polymarket's three back-end services (Gamma, CLOB, Data API) behind a single workflow-shaped surface. Built for one journey: **fetch a market → quote an FOK order → place it (gated) → view positions**.

> **Status:** prototype for a take-home assignment. Not production-ready.
> Polygon mainnet. `signatureType: 2` (Polymarket Safe proxy) only. FOK only.

Quickstart

> **Note:** `@poly-unified/sdk` is not published to npm yet (this is a prototype). The install line below shows what consumption would look like **once published**. For now, clone this repo and use it locally: `npm install` at the repo root, then run the smoke or examples via `npm run smoke` / `npm run example:quote` / etc. See Running the smoke test below.

```
# Available after npm publish:
npm install @poly-unified/sdk viem @polymarket/clob-client-v2
```

```ts
import { PolymarketClient } from "@poly-unified/sdk";

const client = await PolymarketClient.create({
  privateKey: process.env.PRIVATE_KEY as `0x${string}`,
  funderAddress: process.env.FUNDER_ADDRESS as `0x${string}`,
  enableLiveTrading: false, // dryRun bypasses; live submission needs this true
});

// 1. Fetch a market.
const market = await client.markets.get({ slug: "will-something-happen-by-2026" });

// 2. Quote an FOK market order.
const quote = await client.orders.quote({
  market: market!,
  outcome: "Yes",
  side: "BUY",
  amount: 1, // USDC for BUY, shares for SELL
});

// 3a. Dry-run: signs the EIP-712 payload but does NOT submit.
const dryRun = await client.orders.place({
  market: market!,
  outcome: "Yes",
  side: "BUY",
  amount: 1,
  dryRun: true,
});

// 3b. Live submission requires literal `confirm: true` AND `enableLiveTrading: true`.
// const receipt = await client.orders.place({
//   market: market!, outcome: "Yes", side: "BUY", amount: 1,
//   confirm: true, // literal type, rejects `confirm: someBoolean`
// });

// 4. View positions.
const positions = await client.positions.list();
```

Or stitch all four steps with `runWorkflow`:

```ts
const result = await client.runWorkflow({
  slug: "...",
  outcome: "Yes",
  side: "BUY",
  amount: 1,
  dryRun: true,
});
console.log(result.kind); // 'dryRun' | 'live'
```

API surface

Resources (Stripe-style namespace)

Method	Underlying service	Purpose
`client.markets.get({ slug, conditionId?, marketId? })`	Gamma	Slug → normalized `Market`. Throws `MarketAmbiguityError` on multi-match.
`client.markets.getByConditionId(id)`	Gamma	Direct condition-ID lookup.
`client.markets.list(query?)`	Gamma	Browse with filters.
`client.orders.quote({ market, outcome, side, amount, slippageBps? })`	CLOB	FOK quote with avg + worst-price + fee estimate.
`client.orders.place(args)`	CLOB	Place or dry-run an FOK order. See guardrails below.
`client.positions.list({ user?, market?, eventId? })`	Data API	Positions for a user (defaults to funder).
`client.runWorkflow(input)`	All three	Stitches the four-step journey end-to-end.

Escape-hatch sub-clients (off-journey use)

```
import { GammaClient, ClobAdapter, DataApiClient } from "@poly-unified/sdk";

// Direct sub-client access for cancel, getOpenOrders, list events, etc.
const gamma = new GammaClient();
const events = await gamma.list({ active: true, limit: 50 });
```

Live-trading guardrails

`client.orders.place` enforces a **two-gate** policy:

Gate	Question	Enforced by
`confirm: true` (literal type)	"Does this code intend to trade?"	TypeScript compile time: rejects `confirm: false`, missing confirm, or `confirm: someBoolean`

Gate	Question	Enforced by
`enableLiveTrading: true`	"Is this environment authorized to trade?"	Runtime check on the client config / `POLYMARKET_ENABLE_LIVE_TRADING` env var
`dryRun: true` mode	"Skip both, sign for inspection only"	Compile-time discriminated union; bypasses both gates

```
type PlaceOrderArgs =
  | { dryRun: true;  confirm?: never; ... }   // dryRun mode
  | { dryRun?: false; confirm: true;  ... };  // live mode (literal true required)
```

Live submission requires **both gates**. Dry-run requires neither.

Running the smoke test

```
cp .env.example .env
# Fill PRIVATE_KEY, FUNDER_ADDRESS, MARKET_SLUG, OUTCOME, SIDE, AMOUNT.
npm run smoke
```

The smoke runs the read paths (Gamma + CLOB + Data API) plus a `dryRun` order sign. **It does not spend money:** `enableLiveTrading` is forced false inside the script regardless of env.

Examples

Script	What it does
`npm run example:quote`	Fetch market + compute quote. Read-only.
`npm run example:dry-run`	Full workflow with `dryRun: true` . Read-only.
`npm run example:place`	**Live trade:** requires `POLYMARKET_ENABLE_LIVE_TRADING=true` .

CLI flags (all examples + smoke)

Every runnable script accepts CLI flags that override `.env` values per invocation. Wallet secrets (`PRIVATE_KEY` , `FUNDER_ADDRESS`) still come from `.env` , never from CLI, for safety.

```
# Override the market without editing .env
npm run smoke -- --slug=us-x-iran-diplomatic-meeting-by-329

# Disambiguate an event slug
npm run example:quote -- --condition-id=0xbbc6... --side=BUY --amount=2

# Force dry-run on the live-trade example as a safety override
npm run example:place -- --dry-run

# Higher slippage for an illiquid market
npm run example:dry-run -- --slippage=500 --amount=10

# Help for any script
npm run smoke -- --help
npm run example:place -- --help
```

Precedence: **CLI flag > `.env` value > default**.

Available flags: `--slug`, `--condition-id`, `--market-id`, `--outcome`, `--side`, `--amount`, `--slippage`, `--dry-run` (place-order only), `--help`.

Documentation

- `docs/design.md` : design rationale, tradeoffs, what's deferred.
- `docs/ai-usage.md` : how AI/coding agents were used to build this.
- `docs/reference.md` : long-form personal reference with full attribution to Stripe / CCXT / viem / Hyperliquid / `polymarket-kit` (the prior art that inspired specific design choices).
- `docs/plans/2026-04-29-001-feat-poly-unified-sdk-plan.md` : the implementation plan.

Help on any runnable script

Each runnable script supports `--help`:

```
npm run smoke -- --help
npm run example:quote -- --help
npm run example:dry-run -- --help
npm run example:place -- --help
```

The help output enumerates flags, validates inputs, and shows examples specific to each script.

Diagnostics: when smoke fails

If `npm run smoke` fails, the `diag:*` scripts walk you through every layer that could be broken, in order:

```
# 1. Is your funder Safe actually deployed on Polygon?
npm run diag:funder

# 2. Does PRIVATE_KEY's EOA own that Safe?
npm run diag:owner

# 3. Read existing CLOB API credentials (or create new ones)
npm run diag:creds

# 4. Disambiguate an event slug into a specific market
npm run diag:markets us-x-iran-permanent-peace-deal-by

# 5. Verify the named-export sub-clients work standalone (no facade)
npm run diag:subclients
```

These were built during the actual integration debugging that made this SDK work. They catch the most common setup issues (Safe not deployed, wrong wallet, stale CLOB creds, ambiguous slugs).

Development

```
npm install
npm test          # vitest, no network, 153 tests
npm run typecheck # strict TypeScript
npm run build     # emit dist/
```

Known limits (deferred deliberately)

- `signatureType: 2` only (override accepted but only 2 is path-tested).
- FOK only: no GTC/GTD/FAK, batch orders, cancellations, or post-only.
- No WebSockets, no rate-limit retry framework, no balance preflight.
- Browser/CORS not supported (Node ≥ 20 only).

For anything outside the journey, drop down to the named sub-clients (`GammaClient` , `ClobAdapter` , `DataApiClient`) or the underlying `@polymarket/clob-client-v2` directly.